

Deep Learning for NLP

Student name: *Georgios Xydias*
ID: 7115152400007

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025-2026*

Classical ML for Response Clarity Classification

Contents

1	Abstract	2
2	Data processing and analysis	2
2.1	Dataset Description	2
2.2	Exploratory Data Analysis	2
2.3	Text Preprocessing	4
2.4	Input Construction	4
2.5	Data Partitioning	5
3	Text Representations	5
3.1	TF-IDF Features	5
3.2	Word Embeddings	5
4	Experiments	5
4.1	Baseline Models (v0)	6
4.2	Addressing Class Imbalance (v1)	6
4.3	Representation and Classifier Tuning (v2)	6
4.4	Joint Hyperparameter Search (v3)	7
4.5	Results Summary	7
5	Evaluation	8
5.1	Learning Curves	8
5.2	Confusion Matrix	9
5.3	Performance per Class	10
5.4	Test Set Performance	10
6	Discussion	11
6.1	Results Analysis	11
6.2	What Did Not Work	12
6.3	Submitted Configuration	13
6.4	Future Work	13

1. Abstract

This report presents our approach to response clarity classification in political question-answer pairs, as part of SemEval 2026 Task 6 [7]. The dataset, introduced by [6], consists of political interviews with U.S. presidents, where each answer is labeled as *Clear Reply*, *Ambivalent*, or *Clear Non-Reply*. We implement and compare two classical machine learning approaches using scikit-learn [5]: TF-IDF features with Logistic Regression, and pre-trained word embeddings (Word2Vec [3] and GloVe [4]) with Logistic Regression. We apply text preprocessing, address class imbalance, and iteratively tune both text representations and classifier hyperparameters across multiple experimental configurations. Among the explored configurations, our best Kaggle submission uses the GloVe embedding model, which better balances validation performance and generalization, achieving a weighted F1 of 0.58. Our approach follows the methodology outlined in [2] and the relevant chapters of [1]. All implementation details, additional figures, and intermediate results for every experiment presented in this report are available in the accompanying Jupyter notebook.

2. Data processing and analysis

2.1. Dataset Description

We used the dataset introduced by [6] for SemEval 2026 Task 6 [7], which consists of political interviews with U.S. presidents. Each record contains the original journalist turn (`interview_question`), the politician’s response (`interview_answer`), annotation information, metadata (e.g., title, date, president, URL), and auxiliary fields such as GPT-3.5 generated summaries. Because a single journalist turn may contain multiple questions, the dataset provides a segmented question field containing exactly one question. The core supervised instances in this work are therefore (`question`, `interview_answer`) pairs. The prediction target is `clarity_label`, a three-class label: *Clear Reply*, *Ambivalent*, and *Clear Non-Reply*. The dataset is available on Hugging Face under the identifier `ailsntua/QEvasion`, and provides two splits: 3,448 training samples and 308 test samples.

2.2. Exploratory Data Analysis

Before building any model, we performed an exploratory data analysis (EDA) to better understand the dataset. We examined the class distribution, text length statistics, president distribution, and word frequency patterns per class.

The class distribution is highly imbalanced: *Ambivalent* accounts for 59% of the training samples, *Clear Reply* for 31%, and *Clear Non-Reply* for only 10%. This imbalance is important, as a basic model would simply predict the majority class and achieve high accuracy only for this case while failing to detect minority classes. The class distribution is also consistent between the training and test splits.

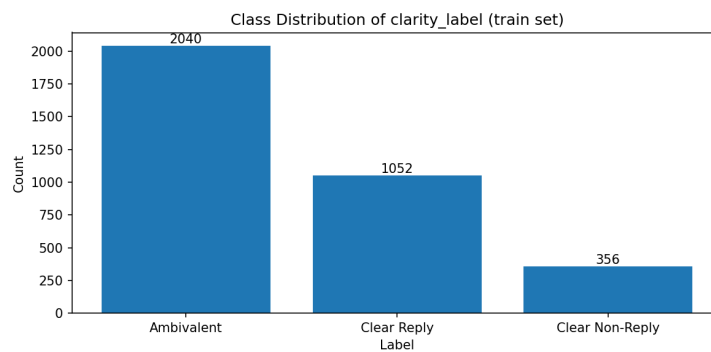


Figure 1: Class distribution in the training set.

Text length analysis revealed a meaningful pattern: *Ambivalent* answers have the highest mean word count (331 words), followed by *Clear Reply* (272 words), and *Clear Non-Reply* (137 words). This suggests that ambivalent answers tend to be the longest, as politicians may use verbosity as a strategy when providing only a partial or indirect response. Clear Non-Reply answers, by contrast, are the shortest, suggesting that evasions are typically brief.

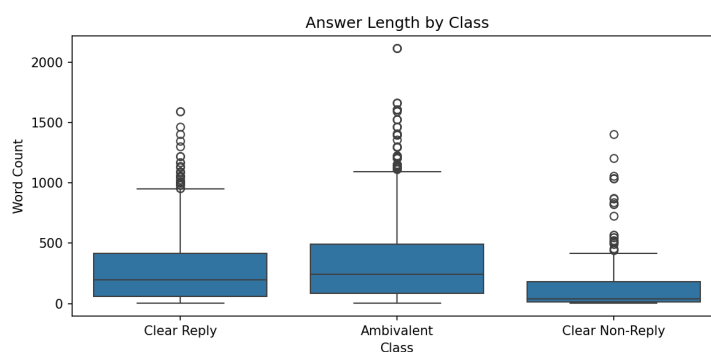


Figure 2: Answer length distribution per class.

Question length, on the other hand, is short and nearly identical across all three classes (mean 12–15 words), confirming that question length carries no discriminative signal for clarity classification.

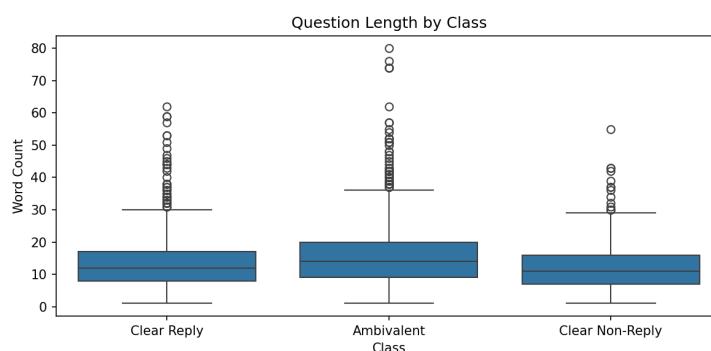


Figure 3: Question length distribution per class.

We also examined the president distribution to check whether certain presidents

are associated with specific clarity patterns. The analysis showed that all presidents follow the same distribution: *Ambivalent* dominates across the board, *Clear Reply* ranks second, and *Clear Non-Reply* is the least frequent in all cases. Since the class imbalance pattern is consistent across all presidents, president identity is not a reliable signal for clarity classification. This is further supported by the fact that the `president` column contains only `None` values in the test set, making it unusable as a feature.

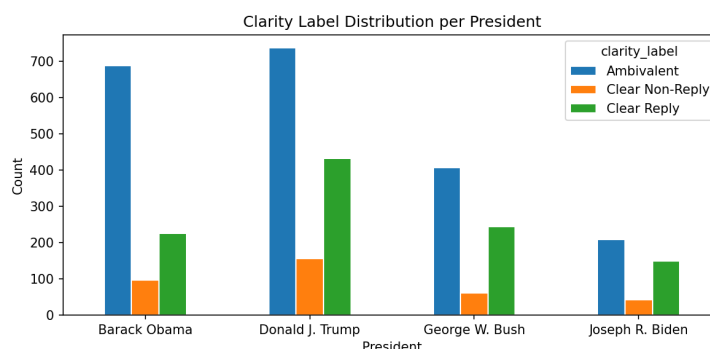


Figure 4: Class distribution per president.

Word frequency analysis showed that the most frequent words are largely shared across all three classes, indicating that word frequency alone is insufficient to distinguish between them. This motivated the use of more expressive features such as bigrams and trigrams, as discussed below.

2.3. Text Preprocessing

We applied a simple but effective preprocessing pipeline to all text fields. First, we removed the prefix `Q.` that appears at the start of some question fields, as it does not add a meaningful content. We also removed bracketed transcription annotations such as `[laughter]` and `[applause]`, which are metadata rather than speech. Then, all text was converted to lowercase so that the same word was treated identically regardless of how it was written. Next, we applied a regular expression to remove all non-alphanumeric characters, replacing them with spaces. Finally, extra whitespace was removed. No lemmatization or stemming was applied, as these did not improve performance in our experiments. Stopword removal was also explored but consistently hurt performance, and is discussed in Section 6.2.

2.4. Input Construction

Each model input is constructed by concatenating the preprocessed *question* and *interview_answer* fields with a separator token:

question [SEP] interview_answer

The separator is added after preprocessing so that the brackets survive the cleaning step intact. This design choice was motivated by the nature of the task: clarity could only be assessed relative to the question asked. We also explored treating the question and answer as separate feature vectors, but this approach performed worse, confirming that a joint representation is more informative (see Section 6.2).

2.5. Data Partitioning

We split the 3,448 training samples into a training set (85%) and a validation set (15%) using stratified sampling, resulting in 2,930 training and 518 validation samples. Stratified splitting ensured that the class distribution is preserved in both splits, which is critical given the severe class imbalance. The test set (308 samples) was kept completely untouched until final prediction. A fixed random seed (SEED=42) was used throughout all experiments to ensure reproducibility.

3. Text Representations

3.1. TF-IDF Features

We represented each input text using the Term Frequency-Inverse Document Frequency (TF-IDF) [1], implemented via `TfidfVectorizer` from `scikit-learn` [5]. TF-IDF assigns a weight to each word based on how frequently it appears in a document relative to how common it is across all documents (corpus). Words that are frequent in a specific document but rare across the corpus receive higher weights, making them more informative for classification.

The vectorizer converts each input text into a sparse vector, where each dimension corresponds to a unique word or n-gram in the vocabulary. With unigrams only, the vocabulary contains 13,501 features. Extending to bigrams expands it to 163,825, and including trigrams further expands it to 502,210, as each additional n-gram captures a unique multi-word combination.

3.2. Word Embeddings

As an alternative to TF-IDF, we also explored dense word embedding representations, where each word is mapped to a fixed-size continuous vector that captures semantic meaning [1]. We experimented with two pre-trained embedding models:

- **Word2Vec** [3]: trained on the Google News corpus (100 billion words), producing 300-dimensional vectors.
- **GloVe** [4]: trained on Wikipedia and Gigaword (6 billion tokens), also producing 300-dimensional vectors.

For both models, each input text was represented by averaging the vectors of all its words (mean pooling). Words not found in the pre-trained vocabulary were ignored. The result is a single 300-dimensional dense vector per input sample, a fixed-size representation regardless of text length.

4. Experiments

Our experimental methodology follows an iterative refinement process: each version builds on the findings of the previous one, addressing its identified weaknesses. We track overfitting or underfitting at each step using learning curves and train/validation accuracy, and select the best configuration based on macro F1 to guide the next round of changes.

4.1. Baseline Models (v0)

We started with two baseline models using default settings to establish a performance reference point. For TF-IDF, we used `TfidfVectorizer` with default parameters (unigrams only) and a standard Logistic Regression classifier with no class weighting. For word embeddings, we used Word2Vec mean pooling with the same default Logistic Regression setup. Both models were evaluated using macro F1 score on the validation set, which treats all three classes equally regardless of their frequency.

The baseline results revealed two key weaknesses: both models predict almost exclusively *Ambivalent*, achieving very low recall on *Clear Reply* and *Clear Non-Reply* (recall of 0.26 and 0.15 for TF-IDF, 0.11 and 0.04 for Word2Vec) and poor overall macro F1 (0.45 for TF-IDF and 0.34 for Word2Vec), confirming that default settings are insufficient for imbalanced classification tasks.

4.2. Addressing Class Imbalance (v1)

To address the class imbalance identified in v0, we enabled `class_weight="balanced"` in Logistic Regression for both models. This automatically adjusts the loss function to penalize misclassifications of minority classes more heavily, proportional to their inverse frequency [5].

This single change produced a significant improvement: TF-IDF macro F1 increased from 0.45 to 0.58, and Word2Vec macro F1 from 0.34 to 0.47. However, the learning curves revealed that overfitting persisted for TF-IDF (large train/val gap), while Word2Vec remained in an underfitting regime (both curves low and close together), motivating further tuning of the representations and classifier hyperparameters in v2.

4.3. Representation and Classifier Tuning (v2)

Building on v1, we tuned both the representation and classifier hyperparameters for each model. The two pipelines followed different strategies.

TF-IDF + LR. We tuned the TF-IDF vectorizer sequentially, fixing each parameter before moving to the next. First, we searched over `max_features` $\in \{\text{None}, 3000, 5000, 10000, 12000\}$, finding that an unconstrained vocabulary (None) performed best (F1: 0.582). Next, we tuned `ngram_range` $\in \{(1,1), (1,2), (1,3)\}$, with trigrams yielding the best result (F1: 0.610). Finally, we tuned `sublinear_tf`, `min_df`, and `max_df` jointly over four configurations, with the best being `sublinear_tf=True`, `min_df=3`, `max_df=0.9` (F1: 0.623). On top of this fixed representation, we performed a grid search over Logistic Regression hyperparameters (`C`, `solver`, `penalty`), arriving at `C=1`, `solver=lbfgs`, `penalty=l2`, yielding a final validation macro F1 of 0.623. The learning curves still showed a large train/val gap, indicating persistent overfitting, motivating the joint search in v3.

GloVe + LR. For the embedding model, we first compared Word2Vec and GloVe under the same setup, finding that GloVe outperformed Word2Vec (F1: 0.483 vs. 0.467) and was selected as the embedding for all subsequent experiments. We then tuned the Logistic Regression hyperparameters on GloVe representations, with the best configuration being `C=10`, `solver=liblinear`, `penalty=l2`, yielding a validation macro F1 of 0.494. Unlike TF-IDF, the learning curves for GloVe showed train and validation F1 close together throughout, indicating that the model generalizes well but has a low

performance ceiling.

4.4. Joint Hyperparameter Search (v3)

In v2, TF-IDF and Logistic Regression hyperparameters were tuned sequentially: first the vectorizer parameters were fixed one by one, then the classifier was tuned on top. This strategy is fast and interpretable, but is suboptimal in two ways. First, within the vectorizer tuning itself, each parameter is fixed before moving to the next. For example, `max_features` was tuned assuming unigrams, but the optimal vocabulary size may differ when trigrams are used instead, since the feature space expands dramatically. Second, once the best vectorizer configuration is fixed, tuning the Logistic Regression on top may also be suboptimal. For example, the best regularization strength `C` may shift depending on the vocabulary size and `n-gram` range, since a larger, noisier feature space may require stronger regularization. Tuning them independently means the best joint combination may never be found. In v3, we therefore searched over all TF-IDF and Logistic Regression hyperparameters simultaneously using `itertools.product` to enumerate all combinations. Based on findings from v2, `sublinear_tf=False` and `penalty=l1` were excluded as they never outperformed the alternatives in any previous trial. The search space is shown in Table 1.

Parameter	Values Tried	Description
<code>ngram_range</code>	(1,2), (1,3)	Include bigrams or trigrams
<code>max_features</code>	None, 10000, 50000	Vocabulary size limit
<code>sublinear_tf</code>	True	Apply log normalization to term frequency
<code>min_df</code>	2, 3	Ignore words appearing in fewer than N documents
<code>max_df</code>	0.90, 0.95	Ignore words appearing in more than N% of documents
<code>C</code>	0.5, 1, 5, 10	Regularization strength
<code>solver</code>	<code>lbfgs</code> , <code>liblinear</code>	Optimization algorithm
<code>penalty</code>	<code>l2</code>	Type of regularization

Table 1: Joint hyperparameter search space in v3.

We ran the joint search under two strategies to assess reliability. First, we searched on the fixed validation set (192 combinations), which found the best configuration to be `ngram_range=(1,2)`, `min_df=2`, `max_df=0.9`, `C=5`, `solver=liblinear`, `penalty=l2`, achieving a validation macro F1 of 0.631. Second, we repeated the same search using 5-fold StratifiedKFold cross-validation (960 fits), which selected `ngram_range=(1,3)`, `min_df=2`, `max_df=0.95`, `C=1`, `solver=lbfgs`, `penalty=l2`, with a CV macro F1 of 0.547 and a validation macro F1 of 0.620. The motivation for running both was to check whether the fixed-split result was inflated by overfitting to that specific validation fold. The fixed-split search achieved a slightly higher validation macro F1 (0.631 vs. 0.620), but the CV-selected parameters yielded a marginally smaller train/val gap in the learning curves, suggesting better generalization.

4.5. Results Summary

Table 2 summarizes the validation set performance across all experiments.

Model	Version	Accuracy	Precision	Recall	Macro F1
TF-IDF + LR	v0 (baseline)	0.64	0.64	0.44	0.453
Word2Vec + LR	v0 (baseline)	0.61	0.57	0.37	0.336
TF-IDF + LR	v1 (class weighting)	0.64	0.57	0.60	0.582
Word2Vec + LR	v1 (class weighting)	0.53	0.47	0.52	0.467
TF-IDF + LR	v2 (repr + LR tuning)	0.66	0.62	0.62	0.623
GloVe + LR	v2 (LR tuning)	0.58	0.51	0.50	0.494
TF-IDF + LR	v3 (joint, fixed val)	0.68	0.66	0.61	0.631
TF-IDF + LR	v3 (joint, CV)	0.66	0.63	0.61	0.620

Table 2: Results summary across all experiments on the validation set.

The largest single improvement came from adding class balancing in v1 (TF-IDF: 0.453 $\rightarrow$ 0.582), confirming that class imbalance was the dominant challenge. Representation and classifier tuning in v2 provided a further gain, while the joint search in v3 yielded a marginal additional improvement.

5. Evaluation

5.1. Learning Curves

We plotted learning curves to examine how training and validation macro F1 change as the number of training samples increases, using them to detect overfitting and underfitting at each stage of development.

TF-IDF + LR. Figure 5 compares the baseline (v0) and the final (v3 CV) TF-IDF models. In v0, there is a large and persistent gap between the training and validation curves, with the validation F1 remaining flat and low throughout, indicating severe overfitting. In v3 CV, the gap is noticeably smaller and the validation curve shows a slow but consistent upward trend, confirming that class balancing and joint hyperparameter tuning significantly reduced overfitting. The still-increasing validation line also suggests that additional training data could improve performance further.

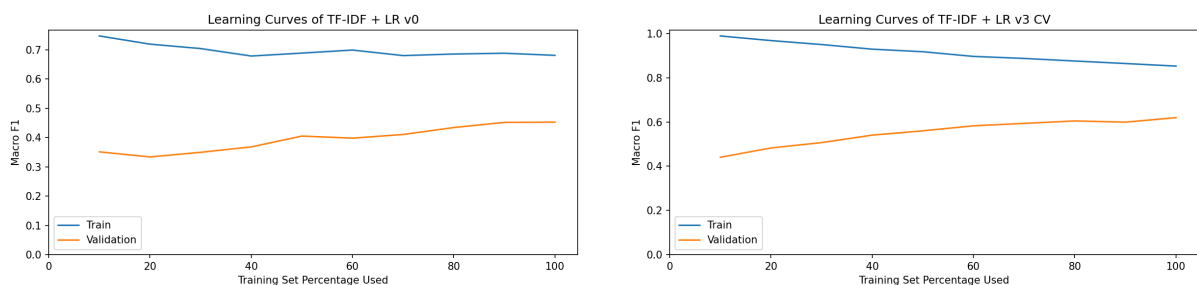


Figure 5: TF-IDF + LR learning curves: baseline (v0, left) vs. best model (v3 CV, right).

GloVe + LR. Figure 6 compares the Word2Vec v1 and GloVe v2 embedding models. In contrast to TF-IDF, both curves remain close together throughout training in both versions, indicating that the embedding models do not suffer from overfitting. However, the validation F1 stabilizes early and at a relatively low level, suggesting that the

mean pooling representation has a limited capacity to capture the discriminative signal needed for this task regardless of tuning.

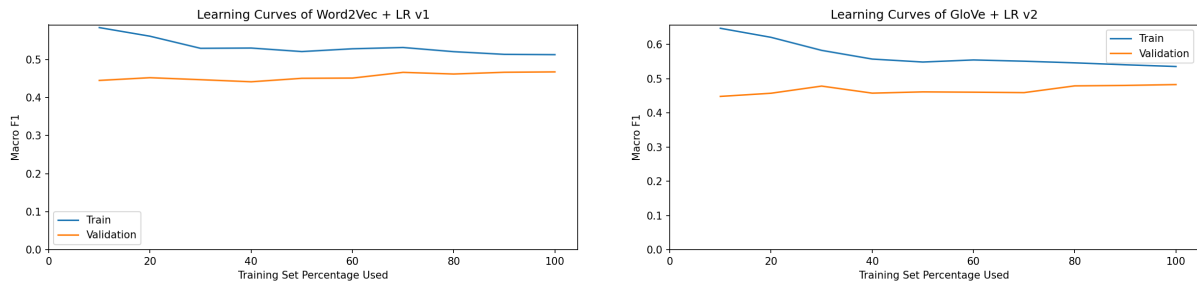


Figure 6: Embedding model learning curves: Word2Vec v1 (left) vs. GloVe v2 (right).

5.2. Confusion Matrix

We examined the confusion matrices across both model families to understand where misclassifications occur and how tuning improved performance.

TF-IDF + LR. Figure 7 compares the baseline (v0) and the v3 CV model. In v0, the model almost entirely predicts *Ambivalent*: out of 54 true *Clear Non-Reply* samples only 8 are correctly classified, and out of 158 true *Clear Reply* samples only 41 are correct. After tuning, v3 CV correctly identifies more samples from all three classes. *Clear Non-Reply* improved from 8 to 30 correct predictions and *Clear Reply* from 41 to 84. However, *Ambivalent* recall dropped slightly, from 280 to 229 correct predictions, as the model now allocates more attention to the minority classes. *Clear Reply* remains the hardest class, with 70 out of 158 samples misclassified as *Ambivalent*. *Clear Non-Reply* also struggles, with 16 out of 54 samples misclassified as *Ambivalent* and 8 as *Clear Reply*.

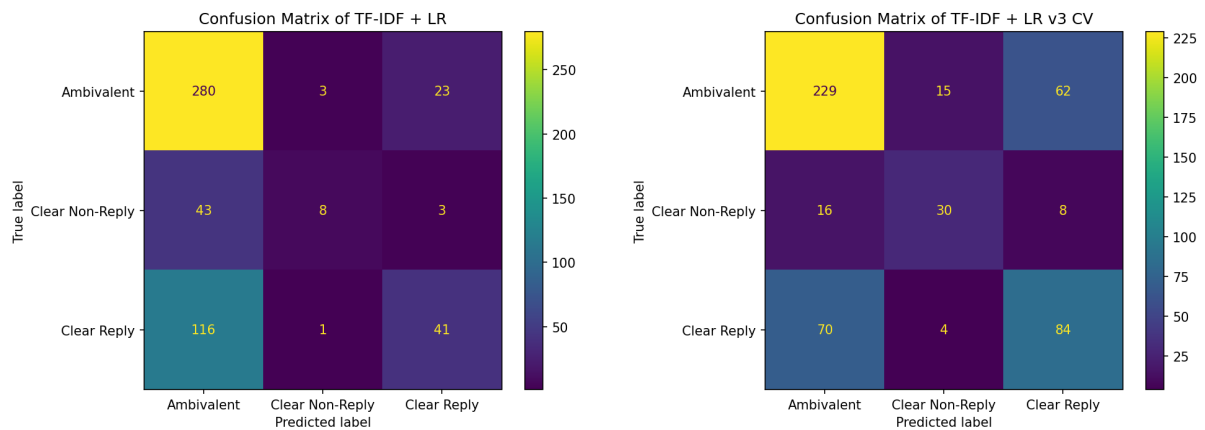


Figure 7: Confusion matrices of TF-IDF + LR: v0 (left) vs. v3 CV (right).

GloVe + LR. Figure 8 compares the Word2Vec v0 baseline and the GloVe v2 model. In v0, the model almost entirely predicts *Ambivalent*: only 2 out of 54 *Clear Non-Reply* and 17 out of 158 *Clear Reply* samples are correctly classified. After switching to GloVe and tuning the classifier, v2 shows improvement across all minority classes: *Clear Non-Reply* improved from 2 to 25 correct predictions and *Clear Reply* from 17 to 41. As with TF-IDF, *Ambivalent* recall dropped from 299 to 237, as the model redistributes attention

toward the minority classes. *Clear Reply* remains the hardest class, with 110 out of 158 samples misclassified as *Ambivalent*.

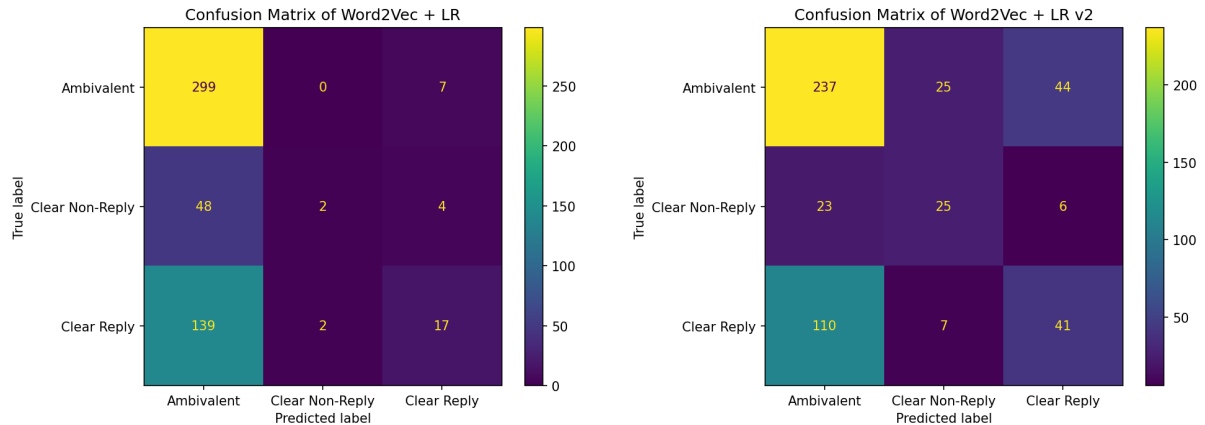


Figure 8: Confusion matrices of embedding models: Word2Vec v0 (left) vs. GloVe v2 (right).

5.3. Performance per Class

Table 3 shows the per-class precision, recall, and F1 score for TF-IDF + LR v3 CV and GloVe + LR v2 on the validation set.

Class	TF-IDF + LR v3 CV			GloVe + LR v2		
	P	R	F1	P	R	F1
Ambivalent	0.73	0.75	0.74	0.64	0.77	0.70
Clear Non-Reply	0.61	0.56	0.58	0.44	0.46	0.45
Clear Reply	0.55	0.53	0.54	0.45	0.26	0.33
Macro avg	0.63	0.61	0.62	0.51	0.50	0.49

Table 3: Per-class performance on the validation set.

Both models follow the same pattern: *Ambivalent* achieves the highest F1 in both cases, reflecting its dominance in the training data. *Clear Reply* is the hardest class for both models, with TF-IDF achieving F1 of 0.54 and GloVe only 0.33. *Clear Non-Reply* sits in between, with TF-IDF again outperforming GloVe (0.58 vs. 0.45). TF-IDF achieves a better balance across all three classes, while GloVe struggles particularly with *Clear Reply*, predicting it correctly only 26% of the time.

5.4. Test Set Performance

Having evaluated both model families on the validation set, we now report performance on the untouched test set. Throughout all experiments, we used macro F1 as the primary evaluation metric, as it treats all three classes equally regardless of their frequency, assigning each class a weight of 33%. The Kaggle competition, however, uses weighted F1, which averages per-class F1 scores by class size, meaning the majority class *Ambivalent* contributes more to the final score. Optimizing for macro F1 may therefore not always yield the best weighted F1, and vice versa. To account for this, we

evaluated several configurations on the test set, including variants with and without `class_weight="balanced"`, as removing class weighting tends to favor the majority class and can improve weighted F1 at the cost of macro F1.

Model	Macro F1	Weighted F1
TF-IDF v3 CV	0.407	0.562
TF-IDF v3 Fixed	0.388	0.582
TF-IDF v3 Fixed (no class weight)	0.385	0.594
GloVe v2 CV	0.434	0.574
GloVe v2 CV (no class weight)	0.399	0.590

Table 4: Test set performance across evaluated configurations.

The results reveal a clear trade-off: configurations without class weighting achieve higher weighted F1 but lower macro F1, confirming that they improve performance on *Ambivalent* at the expense of the minority classes. Notably, GloVe v2 CV achieves the highest macro F1 on the test set (0.434), outperforming all TF-IDF configurations, despite TF-IDF having higher validation macro F1. This suggests that GloVe generalizes better to unseen data. Based on the stronger generalization of GloVe on the test set, and given that the competition metric is weighted F1, we selected GloVe v2 CV without class weighting for the Kaggle submission, as it achieves the highest weighted F1 among the GloVe configurations (0.590).

6. Discussion

6.1. Results Analysis

Our experiments show a clear progression in performance across all versions. The most significant improvement came from enabling class balancing in v1, which raised the TF-IDF macro F1 from 0.45 to 0.58 and the Word2Vec macro F1 from 0.34 to 0.47. This confirms that addressing class imbalance is the single most important factor in this task, given that *Clear Non-Reply* accounts for only 10% of the training data.

In v2, we performed sequential tuning of the TF-IDF vectorizer and Logistic Regression hyperparameters to get a sense of which parameters perform better, while acknowledging that this strategy is suboptimal since each parameter is fixed before the next is tuned. The joint search in v3 yielded a marginal additional improvement on the fixed validation set (0.631), but the CV-selected configuration achieved 0.620, suggesting that part of the fixed-split gain was due to overfitting to that specific validation fold. Notably, the sequential search in v2 identified trigrams as the best n-gram range, while the joint search in v3 selected bigrams, illustrating how tuning components separately can lead to suboptimal combinations.

On the validation set, TF-IDF outperformed GloVe across all versions. We attribute this to three reasons. First, TF-IDF with bigrams and trigrams captures multi-word phrasing patterns that are lost when word vectors are averaged. Second, the dataset is relatively small (2,930 training samples), where sparse and precise TF-IDF features tend to be more reliable than dense averaged embeddings. Third, mean pooling compresses the entire answer into a single 300-dimensional vector, destroying local phrasing signals that are informative for clarity classification.

On the test set, however, GloVe v2 CV achieved the highest macro F1 (0.434), outperforming all TF-IDF configurations (best: 0.407). This reversal suggests that TF-IDF overfit more to the validation set during tuning, while GloVe's smoother decision boundary generalized better to unseen data.

6.2. What Did Not Work

Several approaches were explored but did not improve performance.

Stopword removal. We explored three strategies: scikit-learn's built-in English stopwords list, a custom list derived from the word frequency analysis in the EDA, and the 50 most frequent words across the corpus. All three hurt performance, confirming that high-frequency words such as "not" and "will" carry discriminative signal for this task and should not be removed.

Answer-to-question length ratio as an extra feature. Motivated by the EDA finding that answer length differs across classes, we computed the ratio of answer word count to question word count and appended it as an extra feature to both the TF-IDF sparse matrix and the GloVe dense vectors. Performance dropped in both cases (TF-IDF: 0.620 $\rightarrow$ 0.618, GloVe: 0.494 $\rightarrow$ 0.490), suggesting that this ratio does not add signal beyond what the representations already capture.

Deduplication. After preprocessing, we detected 81 duplicate inputs in the training set that were invisible in the raw data, as the duplicates differed only in GPT-3.5 generated fields which we do not use as features. Removing them reduced the training set from 2,930 to 2,870 samples. For TF-IDF, deduplication slightly hurt performance (0.620 $\rightarrow$ 0.611), while for GloVe it marginally helped (0.494 $\rightarrow$ 0.503). Given the inconsistent effect, we kept the duplicates in the final training set.

Character n-grams. We replaced word n-grams with character n-grams (`analyzer='char_wb', ngram_range=(3,5)`), which reduced performance, indicating that character-level features are less informative than word-level features for this task.

Separate TF-IDF for question and answer. Instead of concatenating question and answer with [SEP], we vectorized them separately and combined the resulting sparse matrices. Performance dropped, confirming that joint representation is more informative.

Feature scaling on GloVe. Applying `StandardScaler` to the GloVe embeddings before classification reduced performance, suggesting that the embedding dimensions are already on a comparable scale.

Max pooling. Replacing mean pooling with max pooling for GloVe embeddings reduced performance, confirming that mean pooling is the better aggregation strategy for this task.

Alternative classifiers. We compared Logistic Regression against Random Forest, SGD Classifier, Ridge Classifier, and Linear SVM on both the TF-IDF v3 CV and GloVe v2 representations. Results are shown in Table 5.

Classifier	TF-IDF v3 CV	GloVe v2
Logistic Regression	0.620	0.494
Linear SVM	0.623	0.496
Ridge Classifier	0.618	0.474
SGD Classifier	0.609	0.338
Random Forest	0.542	0.425

Table 5: Alternative classifiers on TF-IDF v3 CV and GloVe v2 representations.

Linear SVM marginally outperformed Logistic Regression on both representations, while Random Forest was the weakest classifier in both cases. We retained Logistic Regression as it was explicitly required by the assignment and is more interpretable.

Full implementation details, additional results, and figures for all experiments discussed in this section are available in the accompanying notebook.

6.3. Submitted Configuration

We submitted GloVe v2 CV without class weighting, which achieves the highest weighted F1 of 0.590 on the validation set, aligning with the Kaggle competition metric. On the official leaderboard, the submission achieved a weighted F1 of 0.58.

6.4. Future Work

Several directions could improve performance beyond what was achieved here. Collecting more training data could directly address the data limitation observed, especially in the learning curves, where the still-increasing validation line suggests the model has not yet reached its full potential. Replacing Logistic Regression with neural architectures such as LSTMs combined with word embeddings could preserve word order and context, addressing the main limitation of mean pooling. Finally, transformer-based models such as BERT could capture deeper contextual relationships between question and answer that classical methods miss, potentially offering the largest performance gain.

References

- [1] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2026. Online manuscript released January 6, 2026.
- [2] Manolis Koubarakis. Course slides: Introductory concepts of machine learning, regression, word vectors, word2vec, 2026. Artificial Intelligence II Course, Athens University of Economics and Business.
- [3] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [4] Jeffrey Pennington, Richard Socher, and Christopher Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.

- [5] Scikit-learn Developers. Scikit-learn: Machine learning in Python. Accessed: 2026.
- [6] Konstantinos Thomas, Giorgos Filandrianos, Maria Lymperaiou, Chrysoula Zerva, and Giorgos Stamou. “I never said that”: A dataset, taxonomy and baselines on response clarity classification. *arXiv preprint arXiv:2409.13879*, 2024.
- [7] Konstantinos Thomas, Giorgos Filandrianos, Maria Lymperaiou, Chrysoula Zerva, and Giorgos Stamou. CLARITY: SemEval 2026 task 6, 2026.